c:\hpc-23-feb-2026\.\39\eg216.cpp

```
#include<iostream>
#include<unistd.h>
#include<sys/stat.h>
#include<fcntl.h>
using namespace std;
int main()
{
cout<<O_RDONLY<<endl;
cout<<O_RDWR<<endl;
cout<<O_CREAT<<endl;
cout<<O_TRUNC<<endl;
cout<<(O_RDWR | O_CREAT | O_TRUNC)<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\39\eg217.cpp

```
#include<sys/mman.h>
#include<unistd.h>
#include<fcntl.h>
#include<iostream>
#include<string.h>
using namespace std;
int main()
{
const char *data="ABCDEFGHIJ";
int fd;
fd=open("xyz.abc",O_RDWR | O_CREAT | O_TRUNC,0666);
if(fd==-1)
{
cout<<"Unable to create xyz.abc"<<endl;
return 1;
}
if(ftruncate(fd,10)==-1)
{
cout<<"Unable to setup size of xyz.abc"<<endl;
close(fd);
return 1;
}
// MAP_SHARED required of mapping for write operation,
// otherwise kernel space memory contents will not be flushed to disk
char *ptr=(char *)mmap(NULL,10,PROT_READ | PROT_WRITE,MAP_SHARED,fd,0);
memcpy(ptr,data,10);
munmap(ptr,10);
close(fd);
cout<<"File created"<<endl;
return 0;
}
```

c:\hpc-23-feb-2026\.\39\copy_mmap.cpp

```
#include<stdio.h>
#include<sys/stat.h>
#include<sys/mman.h>
#include<unistd.h>
#include<fcntl.h>
#include<string.h>
#include<iostream>
using namespace std;
int main()
{
string source="xyz.abc";
string target="pqr.pqr";
struct stat s;
if(stat(source.c_str(),&s)<0)
{
cout<<"Unable to determine size of "<<source<<endl;
return 1;
}
size_t source_file_size=s.st_size;
int source_fd;
source_fd=open(source.c_str(),O_RDONLY);
if(source_fd<0)
{
cout<<"Unable to open "<<source<<endl;
return 1;
}
char *source_ptr=(char *)mmap(NULL,source_file_size,PROT_READ,MAP_PRIVATE,source_fd,0);
if(source_ptr==MAP_FAILED)
{
close(source_fd);
cout<<"Unable to map source file"<<endl;
// use some fallback technique
return 1;
}
int target_fd;
target_fd=open(target.c_str(),O_RDWR | O_CREAT | O_TRUNC,0666);
if(target_fd<0)
{
munmap(source_ptr,source_file_size);
close(source_fd);
return 1;
}
if(ftruncate(target_fd,source_file_size)<0)
{
cout<<"Unable to setup size of "<<target<<endl;
munmap(source_ptr,source_file_size);
close(source_fd);
close(target_fd);
```

```
return 1;
}
char *target_ptr=(char *)mmap(NULL,source_file_size,PROT_READ | PROT_WRITE,MAP_SHARED,target_fd,0);
if(target_ptr==MAP_FAILED)
{
cout<<"Unable to map "<<target<<endl;
munmap(source_ptr,source_file_size);
close(source_fd);
close(target_fd);
return 1;
}
memcpy(target_ptr,source_ptr,source_file_size);
cout<<"File "<<target<<" created"<<endl;
munmap(source_ptr,source_file_size);
munmap(target_ptr,source_file_size);
close(source_fd);
close(target_fd);
return 0;
}
```

Thinking Machines Classroom Codes